

Verification of Functional Programs

Induction

Andrés Sicard-Ramírez

Universidad EAFIT

Semester 2026-1

Sets Defined by Induction

Sets Defined by Induction

Example

We inductively define the set of natural numbers Nat .

Usual definition.

- (i) $\text{zero} : \text{Nat}$.
- (ii) If $n \in \text{Nat}$ then $\text{succ } n : \text{Nat}$.

Definition using inference rules.

$$\frac{}{\text{zero} : \text{Nat}} \qquad \frac{n : \text{Nat}}{\text{succ } n : \text{Nat}}$$

Sets Defined by Induction

Example

We inductively define the set of natural numbers Nat .

Usual definition.

- (i) $\text{zero} : \text{Nat}$.
- (ii) If $n \in \text{Nat}$ then $\text{succ } n : \text{Nat}$.

Definition using inference rules.

$$\frac{}{\text{zero} : \text{Nat}} \qquad \frac{n : \text{Nat}}{\text{succ } n : \text{Nat}}$$

Remark: In both definitions the fact that Nat is the smallest set generated by the clauses/rules is not stated explicitly.

Structural Induction

Structural Induction

Example

Let P be a unary property on $\mathbf{Nat}$. To prove that $P\ n$ for all $n : \mathbf{Nat}$, we can use **structural induction** for P .

$$\frac{\begin{array}{c} [P\ n] \\ \vdots \\ P\ \text{zero} \quad P\ (\text{succ } n) \end{array}}{P\ n} \text{ (structural induction)}$$

Structural Induction

Example

Let A be a set. We inductively define the lists of elements of A , denoted $\text{List } A$, by the following inference rules:

$$\frac{}{\text{nil} : \text{List } A} \qquad \frac{x : A \quad xs : \text{List } A}{\text{cons } x \ xs : \text{List } A}$$

Structural Induction

Example

Let A be a set. We inductively define the lists of elements of A , denoted $\text{List } A$, by the following inference rules:

$$\frac{}{\text{nil} : \text{List } A} \qquad \frac{x : A \quad xs : \text{List } A}{\text{cons } x \ xs : \text{List } A}$$

Let A be a set and let P be a unary property on $\text{List } A$. To prove that $P \ xs$ for all $xs : \text{List } A$, we can use **structural induction** for P and $\text{List } A$.

$$\frac{\begin{array}{c} [P \ xs] \\ \vdots \\ P \ \text{nil} \quad P \ (\text{cons } x \ xs) \end{array}}{P \ xs} \text{ (structural induction)}$$

Computation Induction

Computation Induction

Example

We recursively define the division by 2 (rounded downwards) function.

$$\lfloor \text{div2} \rfloor : \text{Nat} \rightarrow \text{Nat}$$

$$\lfloor \text{div2} \rfloor \text{ zero} = \text{zero}$$

$$\lfloor \text{div2} \rfloor (\text{succ zero}) = \text{zero}$$

$$\lfloor \text{div2} \rfloor (\text{succ (succ } n)) = \text{succ } (\lfloor \text{div2} \rfloor n)$$

Computation Induction

Example

We recursively define the division by 2 (rounded downwards) function.

$$\lfloor \text{div2} \rfloor : \text{Nat} \rightarrow \text{Nat}$$

$$\lfloor \text{div2} \rfloor \text{ zero} = \text{zero}$$

$$\lfloor \text{div2} \rfloor (\text{succ zero}) = \text{zero}$$

$$\lfloor \text{div2} \rfloor (\text{succ (succ } n)) = \text{succ } (\lfloor \text{div2} \rfloor n)$$

Let P be a unary property on Nat and $\lfloor \text{div2} \rfloor$. To prove that $P n$ for all $n : \text{Nat}$, we can use **computation induction** for P and $\lfloor \text{div2} \rfloor$.

$$\frac{\begin{array}{c} [P \text{ } n] \\ \vdots \\ P \text{ zero} \quad P (\text{succ zero}) \quad P (\text{succ (succ } n)) \end{array}}{P n} \text{ (computation induction)}$$

Computation Induction

Description

Let $f : \sigma \rightarrow \tau$ be a terminating function on the inductive sets σ and τ and let P be a unary property on σ and f . To prove that $P x$ for all $x : \sigma$ by induction on the computation of f it is necessary (p. 6):

- (i) proving that $P e$ for every non-recursive defining equation $f e = e'$, and
- (ii) proving that

$$P \sigma_1 \wedge \dots \wedge P \sigma_n \rightarrow P e$$

for every recursive defining equation

$$f e = \dots f \sigma_1 \dots f \sigma_n$$

where $f \sigma_1, \dots, f \sigma_n$ are all the recursive calls.

Computation Induction[†]

Description

Let $f : \sigma \rightarrow \tau$ be a terminating function on the inductive sets σ and τ and let P be a unary property on σ and f . To prove that $P x$ for all $x : \sigma$ by induction on the computation of f it is necessary (p. 6):

- (i) proving that $P e$ for every non-recursive defining equation $f e = e'$, and
- (ii) proving that

$$P \sigma_1 \wedge \dots \wedge P \sigma_n \rightarrow P e$$

for every recursive defining equation

$$f e = \dots f \sigma_1 \dots f \sigma_n$$

where $f \sigma_1, \dots, f \sigma_n$ are all the recursive calls.

[†]This induction schema is also called “recursion induction”. See, e.g. (Nipkow, Paulson and Wenzel [2002] 2026).

Rule Induction

Rule Induction

Example

We inductively define the inductive relation **Even** on natural numbers by the following rules:

$$\frac{}{\text{zero} : \text{Even}} \qquad \frac{n : \text{Even}}{\text{succ} (\text{succ } n) : \text{Even}}$$

Rule Induction

Example

We inductively define the inductive relation **Even** on natural numbers by the following rules:

$$\frac{}{\text{zero} : \text{Even}} \qquad \frac{n : \text{Even}}{\text{succ} (\text{succ } n) : \text{Even}}$$

Let P be a unary property on **Nat**. To prove that $\text{Even } n \Rightarrow P \ n$, we can use **rule induction** for P and **Even**.

$$\frac{\begin{array}{c} [\text{Even } n, P \ n] \\ \vdots \\ \text{Even } n \quad P \ \text{zero} \quad P \ (\text{succ} (\text{succ } n)) \end{array}}{P \ n} \text{ (rule induction)}$$

Rule Inversion

Example

Rule inversion goes in the opposite direction to rule induction. For example, given **Even** n we can use rule inversion for asking which rules could have been used to derive **Even** n and what constraints these rules impose on n .

References

References

- Tobias Nipkow, Lawrence C. Paulson and Markus Wenzel (2002). Isabelle/HOL. A Proof Assistant for Higher-Order Logic. Vol. 2283. Lecture Notes in Computer Science. Springer. DOI: [10.1007/3-540-45949-9](https://doi.org/10.1007/3-540-45949-9) (cit. on p. 19).
- Tobias Nipkow, Lawrence C. Paulson and Markus Wenzel [2002] (18th Jan. 2026). Isabelle/HOL. A Proof Assistant for Higher-Order Logic. Updated version of (Nipkow, Paulson and Wenzel 2002) matching changes in Isabelle/HOL. URL: <https://isabelle.in.tum.de/dist/Isabelle2025-2/doc/tutorial.pdf> (cit. on p. 13).